

Project Plan for degree project

Template version 1.6 – June 12, 2020

PA2534: KANDIDATARBETE I PROGRAMVARUTEKNIK

March 1, 2026

Title	Automated Remediation in CI/CD: Design and Control of an AI-based Code Repair Agent	
Classification	Software and its engineering, Software maintenance tools	
Student 1	Name	Ahmad Darwich
	E-Mail	ahda23@student.bth.se
	Person nr	030302-7312
	Program	Software Engineering
Student 2	Name	Mouaz Naji
	E-Mail	moap23@student.bth.se
	Person nr	980702-4873
	Program	Software Engineering
Supervisor	Name & title	Ahmad Nauman Ghazi
	E-Mail	nauman.ghazi@bth.se
	Department	Software Engineering

1 Introduction

In the modern landscape of DevOps, software organizations are under constant pressure to achieve high-velocity deployment frequencies while maintaining rigorous system stability. Continuous Integration and Continuous Deployment (CI/CD) pipelines serve as the indispensable backbone of this process, automating the building, testing, and delivery of applications to end-users [1]. By providing a structured path from code commit to production, these pipelines enable teams to iterate rapidly and respond to market demands with agility.

However, as software systems grow in complexity and transition toward distributed architectures, the inherent fragility of these automated pipelines has emerged as a significant operational challenge. CI/CD pipelines frequently fail due to a diverse spectrum of issues, ranging from simple syntax errors and failing unit tests to intricate dependency conflicts and environment mismatches. Despite the high level of automation in the deployment phase, the resolution of these failures remains a stubbornly manual, reactive, and time-intensive endeavor.

Currently, when a build fails, the standard operational procedure requires developers to suspend their primary tasks and engage in "context switching." They must manually parse through voluminous, cryptic execution logs to perform root-cause analysis. This process involves identifying the exact point of failure, understanding the error context, and formulating a corrective patch. This manual intervention creates a substantial bottleneck, significantly increasing the Mean Time to Recovery (MTTR) and diverting valuable engineering resources away from innovation and feature development toward repetitive maintenance work [2].

To address these inefficiencies, the integration of AI-based agents for automated remediation is emerging as a transformative solution. By leveraging recent advancements in Generative AI and Large Language Models (LLMs), these agents have the potential to autonomously detect failures, interpret complex technical logs, and generate targeted code fixes. This promises a shift toward "self-healing" pipelines that can maintain uptime with minimal human oversight.

Nevertheless, the transition from manual remediation to autonomous repair introduces critical technical and operational risks. Generative AI models are known to occasionally "hallucinate" incorrect logic or introduce subtle security vulnerabilities that may bypass traditional linting tools. Consequently, the primary barrier to industry adoption is not the generative capability of the AI, but the fundamental trust in its safety, reliability, and predictability. This research focuses on bridging the gap between raw AI capability and operational safety by exploring how a "code repair agent" can be architected with strict control mechanisms—such as quality gates and audit trails—to support the CI/CD process effectively without introducing technical debt or unsafe changes.

1.1 Ethical, societal and sustainability aspects

The deployment of autonomous agents in software development raises complex ethical questions regarding code ownership and professional accountability. If an autonomous agent introduces a security vulnerability that leads to a catastrophic data breach, establishing a clear chain of liability becomes a non-trivial legal and technical challenge.

From a sustainability perspective, automating routine maintenance can significantly reduce developer burnout and optimize human resource allocation. However, the environmental impact and energy consumption of Large Language Models (LLMs) used for processing logs and generating code must be critically weighed against the efficiency gains and energy saved by reducing prolonged, manual debugging sessions.

2 Aim and objectives

The aim of this thesis is to establish a trusted framework and architectural blueprint for autonomous AI agents in CI/CD pipelines. The goal is to enable automated remediation of routine failures while maintaining strict control over code quality and safety. To achieve this aim, the following objectives have been defined:

- Identify critical risks and necessary "quality gates" (e.g., linting, security scanning) for deploying autonomous repair agents in a production pipeline.
- Develop a detailed architecture blueprint for the agent, including control mechanisms and audit trails to prevent infinite loops or unsafe changes.
- Implement a Proof of Concept (PoC) in Python that demonstrates how an AI agent can interpret build logs and generate targeted, safe code fixes.
- Validate the proposed architecture and prototype through interviews with industry experts to ensure it addresses real-world operational constraints.

3 Research questions

RQ1: How should an AI code repair agent be designed and controlled in a CI/CD pipeline to accurately detect and fix build failures based on logs?

- RQ2: What control mechanisms (e.g., human-in-the-loop, security scanning, audit trails) are required to ensure the agent does not introduce unsafe changes or technical debt?
- RQ3: What are the primary barriers to trust for autonomous remediation agents among software engineers, and how can architectural design mitigate these concerns?

4 Method

The research follows a Design Science Research approach, as we intend to create a practical artifact (the agent architecture and prototype) and evaluate its utility.

- **Phase 1: Architecture Design & Implementation:** Based on literature [3] and best practices, we will design a blueprint for the agent and implement a functional prototype using Python that interfaces with a CI system to read error logs and interact with an LLM API.
- **Phase 2: Data Collection:** We will conduct semi-structured interviews (30–45 min) with 1–3 industry engineers or tech leads to understand real-world failure patterns and trust requirements.
- **Phase 3: Validation:** The proposed architecture and the prototype’s performance will be validated against the feedback gathered from the experts to ensure the solution is viable for real-world use.

5 Expected Outcomes

- A structured architectural blueprint for auditing and controlling AI agents in CI/CD.
- A functional Python-based prototype capable of fixing simple build and test errors autonomously.
- A set of identified "quality gates" and control mechanisms required to ensure safety in automated code repair.

6 Time and activity plan

The project is scheduled to run for approximately 15 weeks, from mid-February to the end of May.

- **Feb 16 – Mar 1:** Inception, literature review, and drafting the initial chapters.
- **Mar 2 – Mar 22:** Design architectural blueprint and develop MVP prototype for syntax errors.
- **Mar 23 – Apr 12:** Implement Quality Gates (security scanning with Bandit, linting with Pylint) and human-in-the-loop logic.
- **Apr 13 – Apr 26:** Expert interviews, demonstration, and qualitative data collection.
- **Apr 27 – May 14:** Final analysis, refinement of blueprint, and thesis submission.

7 Limitations and risk management

The study focuses on Python code to limit technical complexity and uses open-source repositories for testing.

- **Risk:** AI generates incorrect or unsafe code. **Mitigation:** Implementation of strict "quality gates" and a human-in-the-loop approval step.
- **Risk:** Delays in implementation. **Mitigation:** Prioritizing an MVP that handles one error type before expanding.

References

- [1] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [2] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [3] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Wiley, 2011.